TP 1 : Prise en main de Linux & Maîtrise de la Ligne de Commande Bash

HAX712X — Développement Logiciel (M1 SSD)

Bensaid Bilel

2026-09-11

Table des matières

1 Introduction et Réflexes de Survie dans le Terminal **2**
- 1.0.1 Les 4 raccourcis clavier indispensables . . . 2

2 Partie 1 : L'Arborescence et la Navigation (35 min) **2**
- 2.1 Modèle mental : L'arbre inversé . . . 2
- 2.2 Commandes fondamentales d'exploration . . . 3
- 2.3 Exercice 1 : Gymnastique de navigation (À faire directement dans votre terminal) . . . 3

3 Partie 2 : Manipulation de Fichiers & Jokers (Wildcards) (40 min) **4**
- 3.1 Les commandes d'écriture et de gestion . . . 4
- 3.2 Les motifs de filtrage (Wildcards) . . . 4
- 3.3 Préparation de l'environnement de TP . . . 4
- 3.4 Exercice 2 : Rangement automatisé . . . 5

4 Partie 3 : Droits d'Accès et Permissions Linux (35 min) **5**
- 4.1 Décoder les permissions d'un fichier . . . 5
- 4.2 Exercice 3 : Déblocage de script et sécurisation de données . . . 6

5 Partie 4 : Redirections, Filtres et Pipelines (|) (45 min) **6**
- 5.1 Modèle mental : Les tuyaux de données . . . 6
- 5.2 La boîte à outils du Data Analyst en ligne de commande . . . 7
- 5.3 Exercice 4 : Analyse de logs serveur . . . 7

6 Le Défi Récapitulatif (20 min) **7**

7 Pour Aller Plus Loin : Défis Avancés (Optionnel) **8**
- 7.0.1 Défi A : Identification du top 1 des attaquants (Pipeline strict) . . . 8
- 7.0.2 Défi B : Filtrage temporel avec Expressions Régulières (`grep -E`) . . . 8
- 7.0.3 Défi C : Détection de fichiers suspects par date (`find`) . . . 8

1 Introduction et Réflexes de Survie dans le Terminal

Bienvenue dans votre premier TP d'ingénierie logicielle. Le terminal Bash n'est pas un outil d'archiviste : c'est l'interface directe avec le système d'exploitation, indispensable pour le calcul haute performance, la gestion de serveurs distants et le déploiement de modèles de données.

> **i** La grammaire universelle d'une commande Unix
>
> Toutes les commandes Bash obéissent à la même structure :
>
> `nom_commande [-options] [arguments]`
>
> — **La commande** : l'action à exécuter (ex. : `ls`, `cd`, `rm`).
> — **Les options (flags)** : modifient le comportement de la commande. Elles commencent par un tiret `-` pour les options courtes (ex. : `-l`, `-a`, `-h`) ou deux tirets `--` pour les options longues (ex. : `--all`, `--human-readable`). On peut combiner les options courtes : `ls -l -a -h` s'écrit de manière équivalente `ls -lah`.
> — **Les arguments** : la cible sur laquelle s'applique l'action (un nom de fichier, un dossier, une chaîne de texte).

1.0.1 Les 4 raccourcis clavier indispensables

Avant de taper la moindre commande, intégrez ces quatre automatismes :

1. **La touche `Tabulation` (Auto-complétion) :** Ne tapez jamais un nom de fichier ou de dossier en entier. Tapez les deux premières lettres et appuyez sur `Tab`. Le terminal complète le nom automatiquement. Si deux fichiers ont le même début, appuyez deux fois sur `Tab` pour afficher les choix possibles.
2. **Les flèches `Haut` et `Bas` (Historique) :** Inutile de retaper une commande précédente : les flèches permettent de naviguer dans l'historique de vos instructions.
3. **`Ctrl + C` (Arrêt d'urgence) :** Si une commande tourne en boucle infinie ou bloque l'écran, `Ctrl + C` envoie un signal d'interruption immédiat (`SIGINT`) et vous rend la main.
4. **`Ctrl + L` (ou commande `clear`) :** Nettoie l'affichage du terminal sans effacer l'historique de vos actions.

---

2 Partie 1 : L'Arborescence et la Navigation (35 min)

2.1 Modèle mental : L'arbre inversé

Sous Linux, il n'existe pas de disques `C:` ou `D:` comme sous Windows. Tout le système est organisé sous la forme d'un **arbre unique** dont l'origine absolue est la racine, notée `/` (*root*).

Trois symboles spéciaux sont omniprésents : * `/` : la **racine** du système (le sommet de l'arborescence). * `~` (tilde) : votre **répertoire personnel** (*home directory*, ex. : `/home/etudiant`). * `.` : le répertoire **courant** (où vous vous trouvez actuellement). * `..` : le répertoire **parent** (le dossier situé immédiatement au-dessus).

! Chemins Absolus vs Chemins Relatifs

— **Un chemin absolu** commence obligatoirement par `/` ou `~`. Il décrit l'itinéraire depuis la racine du système, quel que soit l'endroit où vous vous trouvez.
  — *Exemple :* `/home/etudiant/Documents/tp1/`
— **Un chemin relatif** ne commence **jamais** par `/`. Il décrit le chemin à emprunter **en partant de l'endroit où vous êtes actuellement**.
  — *Exemple :* si vous êtes dans `Documents/`, le chemin vers le sous-dossier s'écrit simplement `tp1/` ou `./tp1/`. Pour remonter d'un cran, on écrit `../`.

2.2 Commandes fondamentales d'exploration

Commande	Rôle	Exemple type
`pwd`	*Print Working Directory* : affiche le chemin absolu du dossier actuel	`pwd`
`ls`	Liste les fichiers et dossiers visibles	`ls`
`ls -l`	Format long : affiche les droits, propriétaire, taille et date	`ls -l`
`ls -a`	Affiche **tous** les fichiers, y compris les fichiers cachés (qui débutent par `.`)	`ls -a`
`ls -lah`	Format long, tous les fichiers, avec tailles lisibles en Ko/Mo/Go (`-h`)	`ls -lah`
`cd <chemin>`	*Change Directory* : change de répertoire de travail	`cd ..`
`cd ~` ou `cd`	Retourne directement à votre répertoire personnel (*home*)	`cd`
`cd -`	Revient au répertoire où vous étiez juste avant la dernière commande	`cd -`

2.3 Exercice 1 : Gymnastique de navigation (À faire directement dans votre terminal)

1. Ouvrez votre terminal. Tapez `pwd`. Quel est votre emplacement initial ?
2. Déplacez-vous à la racine du système :
   ```
   cd /
   pwd
   ls -l
   ```
   Observez les dossiers fondamentaux d'un système Linux (`/bin`, `/etc`, `/home`, `/var`…).
3. Entrez dans le dossier `etc/` en chemin relatif. Vérifiez avec `pwd`.
4. Remontez d'un cran dans l'arborescence sans utiliser le mot `root`.

5. Retournez dans votre répertoire personnel en une seule commande de deux caractères.
6. Affichez les fichiers cachés de votre répertoire personnel. Repérez les fichiers de configuration débutant par un point (ex. : `.bashrc`, `.profile` ou `.gitconfig`).

---

3 Partie 2 : Manipulation de Fichiers & Jokers (Wildcards) (40 min)

3.1 Les commandes d'écriture et de gestion

- `mkdir dossier` : crée un répertoire (*make directory*). L'option `-p` permet de créer une hiérarchie complète (ex. : `mkdir -p a/b/c`).
- `touch fichier.txt` : crée un fichier vide s'il n'existe pas, ou met à jour sa date d'accès.
- `cp source destination` : copie un fichier (*copy*). Pour copier un dossier entier, l'option récursive `-r` est **obligatoire** : `cp -r dossier_src/ dossier_dest/`.
- `mv source destination` : déplace ou renomme un fichier/dossier (*move*).
- `rm fichier` : supprime définitivement un fichier (*remove*). Attention : **il n'y a pas de corbeille sous Linux**. Toute suppression est irréversible.
- `rm -r dossier` : supprime récursivement un dossier et tout son contenu.
- `cat fichier` : affiche l'intégralité du contenu d'un fichier texte dans la console.

3.2 Les motifs de filtrage (Wildcards)

Pour appliquer une action sur un ensemble de fichiers sans les lister un par un :

- `*` : remplace n'importe quelle chaîne de 0, 1 ou plusieurs caractères. Par exemple `*.csv` correspond à tous les fichiers se terminant par `.csv` et `data_*` à tous les fichiers dont le nom commence par `data_`.
- `?` : remplace **exactement un** caractère unique. Par exemple, `modele_?.py` correspond à `modele_1.py`, `modele_A.py`, mais pas à `modele_10.py`.

3.3 Préparation de l'environnement de TP

Pour dérouler les exercices suivants, téléchargez l'archive du TP. Vous pouvez télécharger l'archive du TP directement :

- En cliquant sur ce lien : Télécharger tp1_linux.tar.gz
- Ou directement dans votre terminal avec la commande `wget` :
  ```
  wget https://bbensaid30.github.io/HAX_dev_provisoire/TP_bash/tp1_linux.tar.gz
  ```

Puis décompressez-la :

```
# Téléchargement et extraction de l'archive
tar -xzvf tp1_linux.tar.gz
cd tp1_linux
pwd
```

3.4 Exercice 2 : Rangement automatisé

Entrez dans le dossier `exercice_1/` :

```
cd exercice_1
ls -l vrac/
```

Le sous-dossier `vrac/` contient un mélange désordonné de fichiers (`.png`, `.txt`, `.py`, `.csv`).

1. Déplacez **en une seule commande** tous les fichiers `.png` vers le dossier existant `documents/images/`.
2. Déplacez **en une seule commande** tous les fichiers `.txt` vers `documents/textes/`.
3. Créez un nouveau répertoire nommé `code/` au même niveau que `documents/`.
4. Copiez (sans le déplacer) le fichier `vrac/script.py` vers ce nouveau dossier `code/`.
5. Supprimez le répertoire `vrac/` devenu inutile avec son contenu restant.

---

4 Partie 3 : Droits d'Accès et Permissions Linux (35 min)

4.1 Décoder les permissions d'un fichier

Sous Unix, chaque fichier et répertoire est protégé par un triplet de droits appliqués à trois niveaux d'utilisateurs. Tapez `ls -l` :

```
-  r w x  r - x  r - -   1   bilel   profs   4096  10 sept.  run.sh
↑
|     |       |         Droits des "Autres" (world / others : o)
|     |             Droits du "Groupe" (group : g)
|               Droits du "Propriétaire" (user / owner : u)
              Type (- = fichier, d = dossier)
```

Chaque triplet se compose de trois attributs :

- `r` (*read* - lecture) : valeur binaire = **4**. Permet de lire le fichier (ou de faire `ls` dans un dossier).
- `w` (*write* - écriture) : valeur binaire = **2**. Permet de modifier ou supprimer le fichier.
- `x` (*execute* - exécution) : valeur binaire = **1**. Permet de lancer le fichier comme un programme exécutable (ou d'entrer `cd` dans un dossier).
- `-` : droit non accordé = **0**.

On peut ainsi coder les droits soit par des lettres (`u+x`, `go-w`), soit par un **chiffre octal** (la somme des valeurs) :

- `755` = `rwxr-xr-x` (le propriétaire peut tout faire ; le groupe et les autres peuvent lire et exécuter).
- `644` = `rw-r--r--` (droit standard d'un fichier texte : modifiable uniquement par son créateur).
- `600` = `rw-------` (fichier privé : seul le propriétaire peut lire et modifier).

La commande pour modifier ces droits est `chmod` (*change mode*).

4.2 Exercice 3 : Déblocage de script et sécurisation de données

Revenez au dossier parent et entrez dans `exercice_2/` :

```
cd ../exercice_2
ls -l scripts/
```

1. Tentez d'exécuter le script de sauvegarde système :
   ```
   ./scripts/backup.sh
   ```
   Quel message d'erreur s'affiche ? Pourquoi ?
2. À l'aide de la commande `ls -l scripts/backup.sh`, identifiez les droits actuels du script.
3. Donnez le droit d'exécution au script en utilisant la notation symbolique. Exécutez-le à nouveau.
4. Observez le fichier `scripts/secret.conf` : il contient une clé d'API critique. Vérifier ses permissions actuelles. Modifiez ses droits en utilisant la **notation octale** pour que vous soyez **le seul** à pouvoir lire et écrire dans ce fichier, en retirant tout droit au groupe et aux autres.
5. Vérifiez le résultat avec `ls -l scripts/secret.conf`. Le motif affiché doit être strictement `-rw-------`.

---

5 Partie 4 : Redirections, Filtres et Pipelines (|) (45 min)

5.1 Modèle mental : Les tuyaux de données

Chaque programme exécuté dans un terminal dispose par défaut de trois flux de communication :

1. **L'entrée standard (`stdin` - canal 0)** : ce que vous tapez au clavier.
2. **La sortie standard (`stdout` - canal 1)** : le résultat produit affiché à l'écran.
3. **La sortie d'erreur (`stderr` - canal 2)** : les messages d'erreurs affichés à l'écran.

Bash permet de détourner ces flux grâce aux **redirections** et au **tube (pipe)** :

— `commande > fichier` : redirige la sortie vers un fichier **en écrasant** son contenu préalable.
— `commande >> fichier` : redirige la sortie **en l'ajoutant à la fin** du fichier (*append*).
— `commande1 | commande2` : connecte directement la sortie de `commande1` à l'entrée de `commande2`. La donnée passe de l'une à l'autre en mémoire sans créer de fichier temporaire sur le disque.

5.2 La boîte à outils du Data Analyst en ligne de commande

- `cat fichier` : affiche tout le fichier.
- `head -n k fichier` / `tail -n k fichier` : extrait les k premières / dernières lignes.
- `grep "motif" fichier` : filtre et n'affiche que les lignes contenant le motif.
  - Option `-i` : insensible à la casse (majuscule/minuscule).
  - Option `-v` : inverse le filtre (affiche les lignes qui **ne contiennent pas** le motif).
  - Option `-c` : compte le nombre de lignes correspondantes.
- `wc -l fichier` : compte le nombre de lignes (*word count line*).
- `cut -d "séparateur" -f numéro` : découpe une ligne selon un délimiteur et extrait la colonne indiquée.
- `sort` : trie les lignes par ordre alphabétique (ou numérique avec `-n`).
- `uniq -c` : élimine les doublons **consécutifs** et compte le nombre d'occurrences.

5.3 Exercice 4 : Analyse de logs serveur

Entrez dans le dossier `exercice_3/logs/` :

```
cd ../exercice_3/logs
```

Le fichier `connexions.log` trace l'activité d'accès à un serveur de base de données.

1. Affichez les 5 premières lignes du fichier pour comprendre sa structure.
2. Affichez uniquement les lignes correspondant à un échec d'authentification (`status=FAILED`).
3. **Pipeline de comptage :** En combinant `grep` et `wc -l` avec un pipe `|`, déterminez en une seule ligne de commande le nombre total de tentatives échouées.
4. **Extraction et sauvegarde :** Isolez toutes les lignes concernant l'utilisatrice `alice` et écrivez le résultat dans un nouveau fichier `rapport_alice.txt`.
5. **Détection d'attaque (Pipeline complet) :** En enchaînant les commandes adéquates avec plusieurs tubes `|` (`grep`, `cut`, `sort`, `uniq -c`), produisez la liste ordonnée des utilisateurs ayant généré des échecs de connexion, avec le nombre précis d'échecs pour chacun.

---

6 Le Défi Récapitulatif (20 min)

Rendez-vous dans le dossier `defi/` :

```
cd ../../defi
```

Trois serveurs (`alpha`, `beta`, `gamma`) contiennent chacun des dizaines de fichiers de logs systèmes `proc_*.log`.

Un incident de sécurité critique a laissé une empreinte unique identifiée par la chaîne de caractères `INDICE_FINAL`.

Votre mission :
À l'aide d'une commande unique utilisant `grep` avec ses options récursives (ou combiné avec `find`), identifiez :

1. Le serveur concerné.
2. Le fichier exact contenant l'empreinte.
3. La valeur secrète du token entre crochets.

---

7 Pour Aller Plus Loin : Défis Avancés (Optionnel)

Cette section s'adresse aux étudiants ayant terminé en avance. Elle mobilise uniquement les commandes vues aujourd'hui, combinées de manière plus poussée.

7.0.1 Défi A : Identification du top 1 des attaquants (Pipeline strict)

En reprenant `exercice_3/logs/connexions.log`, écrivez une commande unique qui affiche **uniquement le nom d'utilisateur** (sans le préfixe `user=` et sans le décompte) ayant cumulé le plus grand nombre d'échecs de connexion (`FAILED`).

— *Indice :* chaînez votre pipeline précédent avec un nouveau `sort` numérique, `tail` (ou `head`), et un dernier `cut`.

7.0.2 Défi B : Filtrage temporel avec Expressions Régulières (`grep -E`)

Affichez uniquement les lignes de connexion enregistrées entre 08h20 et 08h39 inclus.

— *Notion :* l'option `-E` de `grep` active les expressions régulières étendues. Le crochet `[23]` permet de cibler le chiffre 2 ou 3.
— *Indice :* cherchez le motif correspondant à l'intervalle horaire dans la chaîne de date.

7.0.3 Défi C : Détection de fichiers suspects par date (`find`)

Dans le dossier `defi/`, utilisez la commande `find` avec l'option `-mmin` pour localiser automatiquement les fichiers créés ou modifiés au cours des 60 dernières minutes.